Save Custom Game

Documentation

_____

Creator: Lucas Gomes Cecchini

Pseudonym: AGAMENOM

Overview

This document will help you use **Assets Save Custom Game** for **Unity**.

With it you can save your game in several Slots.

It will take a **screenshot** of the game and there are three ways to save in **GameData(.json)**, **LocalLow(.json)** and **PlayerPrefs**.

It also has Auto Save features based on **Time** or **Events**.

Instructions

You can get more information from the Playlist on YouTube:

https://youtube.com/playlist?list=PL5hnfx09yM4JDFFdX1vfpNCLLdLq4ixZA&si=3w1IN0WrpV8lFMrS

Script Explanations

Exception Utility

This utility is designed to **trace where a method call originally came from in your codebase**, mainly for debugging purposes inside your SaveCustomGame system. Instead of manually tracking logs, it inspects the runtime call stack and extracts the most relevant external location (file + line number).
Below is a structured explanation of how everything works.

🧠 Overview of the Script

The main purpose of this class is:

- To inspect the **call stack at runtime**
- To find the **original caller outside internal utilities**
- To return a **readable file path + line number**
- To help debug errors or events more precisely

It intentionally ignores internal helper classes so logs point to meaningful user code.

📦 Namespace: SaveCustomGame

This groups the utility into your save system domain.
Everything inside is related to debugging or saving functionality.

🧰 Static Class: ExceptionUtility

This class is:

- Static → you do not create instances of it
- Pure utility → only provides helper functions
- Focused on debugging information retrieval

Because it is static, you call its method directly without instantiation.

🔍 Method: GetCallingMethodInfo()

This is the core function of the entire script.

🎯 Purpose

It returns a string like:

- “(at Assets/Scripts/Player/PlayerController:42)”

This tells you:

- Which file triggered the call
- Which line in that file executed it

📦 Step-by-step breakdown

1. Stack Trace Creation

The method first creates a snapshot of the current execution chain:

- It captures every method call that led to this point
- It includes file names and line numbers (when debugging symbols exist)

👉 Think of it like a “breadcrumb trail” of function calls.

2. Getting Stack Frames

From the stack trace:

- It extracts individual “frames”
- Each frame represents one method call in the chain

Each frame contains:

- Method name
- Class type

- File path
- Line number

3. Safety Check

Before processing:

- It ensures frames exist
- Prevents null reference issues if stack trace is empty

4. Tracking Internal Calls

A boolean flag is used:

- It detects when a SaveDataUtility method appears in the stack

This is important because:

- Internal system calls should be ignored
- Only external (game/user code) calls matter

5. Looping Through Each Frame

For each frame:

It extracts:

- The method being executed
- The class (type) that owns it

If the class is valid:

- It gets the full class name for comparison

6. Filtering Internal Classes

It checks if the frame belongs to:

- SaveDataUtility → internal save system
- ExceptionUtility → this helper class itself

If NOT either:

That means:

✓ This is external code

✓ This is the real caller we want

So it proceeds to extract file info.

7. Extracting Debug Information

From the selected frame:

It retrieves:

- File name (full system path)
- Line number inside the file

Then it validates:

- File must exist
- Line number must be valid (> 0)

8. Path Simplification (Regex)

The file path is cleaned using a pattern:

- It removes everything before “Assets”

So instead of:

- C:\Users\PC\Project\Assets\Scripts\Player.cs

It becomes:

- Assets\Scripts\Player.cs

This is useful because:

- It matches Unity project structure
- It is easier to read in logs

9. Returning Formatted Result

If everything is valid:
It returns a formatted string like:

- (at Assets/Scripts/Player.cs:120)

This gives precise debug location info.

10. Special Internal Flow Control
The boolean flag is used to manage deeper stack behavior:

- If SaveDataUtility is encountered, it marks it
- If ExceptionUtility is reached after that, it stops scanning

This prevents:

- Over-scanning internal utility layers
- Returning irrelevant stack frames

11. Fallback Return
If nothing valid is found:

- It returns an empty string

Meaning:

- No meaningful external caller was detected

🧪 **How to Use It**
You typically use it in logging or debugging systems.
Example usage scenario:

- You save data
- An error happens
- You log where it came from

Result:

- You immediately see the exact script and line responsible

📊 **Summary of Variables**
stackTrace

- Holds the full execution history
- Contains all method calls leading to this point

frames

- Individual steps in the call stack
- Each represents a function call

foundSaveDataUtility

- Tracks if internal save system appeared
- Helps control when to stop scanning

frame

- Current stack step being analyzed

method

- Function being executed in that frame

declaringType

- Class that owns the method

typeName

- Full name of the class for filtering

fileName

- Physical file path of the script

lineNumber

- Exact line in the file where execution occurred

filePath

- Cleaned version of fileName for Unity readability

🚀 **In Short**

This utility:

- Walks through the call stack
- Skips internal system calls
- Finds the real source of execution
- Returns clean file + line debug info

Save Data Utility

This script is essentially a **central save-data manager + performance layer + debug system + utility toolkit** all combined into one centralized static system.
It is responsible for:

- Reading and writing saved data (floats, ints, strings, bools, vectors)
- Caching everything for performance
- Automatically creating missing data structures
- Logging missing data safely without spam
- Managing scene references
- Handling screenshots
- Converting vector formats
- Providing safe fallback behavior

Below is a structured breakdown of **how it works, variable by variable and method by method**, including usage.

🧠 Overall Concept

Think of this system as a:
"Fast access memory layer on top of Unity save data"
Instead of searching through lists every time, it:

- Builds a dictionary cache
- Uses fast lookup (instant access)
- Creates missing data automatically
- Logs problems only once
- Provides safe fallback values

📦 Cached References Section

📎 saveCustomObject

- Holds the main save file (ScriptableObject)
- This is the **core storage container**

Usage

- Loaded once from Resources
- Reused everywhere after that

If null:

- It is loaded automatically

📎 cachedSceneComponent

- Stores a reference to a scene object component
- Avoids repeated expensive scene searches

Why it matters

- Find() is slow
- So this stores it once and reuses it

📎 itemCache (Dictionary)

What it does:

- Maps itemTag → SaveCustomItem

Why it exists:

Instead of searching like:

- “loop through everything every time”

It becomes:

- instant lookup (O(1) performance)

Example usage concept:

- "PlayerStats" → directly gets its save item

🧪 Debug Control Section

🔧 loggedMissingKeys (HashSet)

Purpose:
Prevents repeated warning spam.

Behavior:

- First time missing data → logs warning
- Next times → ignored

Why:
Avoids console flooding

🔧 LogDefaultUsage()

Purpose:
Logs when default values are used

Inputs:

- itemTag → category of data
- tag → specific value key
- type → data type (float/int/etc.)

Behavior:

1. Builds unique key string
2. Checks if already logged
3. If not → logs warning once
4. Includes call origin using stack trace utility

Output example concept:

- “Using default value for missing key: Player.Health (Type: Float)”

📥 Get Save Object Section

🔧 GetSaveCustomObject()

Purpose:
Main entry point for accessing save data

Flow:

1. If already loaded → return cached object
2. Otherwise:
    - Load from Resources
    - If missing → error log
3. Build cache
4. Return object

🔧 BuildCache()

Purpose:
Builds fast lookup dictionary

Process:

- Clears old cache
- Loops all save items
- Adds them into dictionary

Result:
Fast direct access instead of slow search

Safe Get Methods Section

These are the public safe-read functions.

GetFloat / GetInt / GetString / GetBool / GetVector

Purpose:

Retrieve saved values safely

Behavior pattern:

1. Try to get value using TryGet method
2. If successful → return value
3. If not:
    - Log warning (only once)
    - Return default value

Default fallback behavior:

- float → 0
- int → 0
- string → ""
- bool → false
- vector → (0,0,0,0)

TryGet Methods Section

These are low-level retrieval functions.

TryGetFloat / TryGetInt / etc.

Purpose:

Attempt retrieval without logging or fallback

They call:

A shared internal system called TryFind

TryFind (core retrieval engine)

This is the **heart of reading data**

Inputs:

- itemTag → category (e.g. PlayerData)
- tag → specific value (e.g. Health)
- getList → chooses correct list (float/int/etc.)
- getTag → extracts key name
- getValue → extracts stored value

How it works:

1. Get save object
2. Find item in dictionary (fast)
3. Get correct list (floats, ints, etc.)
4. Loop through entries
5. If tag matches:
    - return value
6. If not found:
    - return false

Key idea:

It avoids searching entire save system repeatedly.

Set Methods Section

These are write/update operations.

SetFloat / SetInt / SetString / SetBool / SetVector

Purpose:

Store or update values in save system
Behavior:
They all call a shared method:
SetValue

🔧 SetValue (core writing engine)
This is the most important write function.

Step-by-step:
1. Ensure save exists
- Loads save object if needed

2. Ensure item list exists
- If null → creates it

3. Find or create item
- Uses dictionary cache
- If missing:
  - Creates new SaveCustomItem
  - Adds it to cache + list

4. Get correct value list
- Example:
  - float list
  - int list
  - vector list

5. Find existing entry
- If exists:
  - update value

6. If not found:
- create new entry
- add to list

Result:
Always safe write (no missing data possible)

🎮 Scene Access Section
🔧 GetComponentSaveCustomInScene()
Purpose:
Get scene object containing save system

Flow:
1. If cached → return
2. Find GameObject by name
3. If missing → log error
4. Get component from object
5. Cache it

Why important:
Avoids repeated expensive scene searches

💾 Save Control Section

📎 SetAutoSave()
Purpose:
Enable or disable autosave system
Behavior:
- Gets save object
- Sets autosaveEnabled flag

📎 SaveEvent()
Purpose:
Triggers save manually
Behavior:
- Gets scene component
- Finds AutoSave system
- Calls SaveAutoGame()

🔄 Vector Conversion Section
📎 Vector4ToQuaternion()
Purpose:
Converts vector data into rotation data

Steps:
1. Build quaternion from vector
2. Normalize it manually
3. If invalid → fallback to identity
4. Log warning if corrupted

Why normalization matters:
Prevents broken rotations in Unity

📸 Screenshot System Section
📎 CaptureScreenshot()
Purpose:
Takes in-game screenshot and stores it in save data

Flow:
1. Validate save + camera
2. Calculate resolution (with pixel limit)
3. Store current render state
4. Create render texture
5. Render camera
6. Read pixels into texture
7. Encode to PNG
8. Store in save system
9. Restore everything
10. Clean memory

Key features:
- Safe memory handling
- No leaks (try/finally cleanup)
- Resolution clamping

📎 RenderScreenshot()
Purpose:

Convert saved PNG bytes back into texture
Behavior:

- Validate data
- Create small texture
- Load image
- Mark non-readable (memory optimization)

🩹 TextureToSprite()

Purpose:
Convert texture into UI sprite
Usage:

- UI icons
- thumbnails
- previews

🚀 Final Summary

This system is built around 5 core principles:

⚡ Performance

- Dictionary caching
- O(1) lookups

🛡 Safety

- Null checks everywhere
- Automatic creation of missing data

🧠 Debuggability

- Stack trace logging
- One-time warnings only

🔄 Flexibility

- Works with multiple data types
- Generic internal system

🎮 Unity integration

- Scene component caching
- Screenshot pipeline
- Resources loading

Create Base Script

This script is not part of gameplay logic itself — it is an **Unity Editor tool that generates ready-made scripts for your Save Custom Game system automatically**.
In simple terms:
It adds menu buttons inside Unity that create full prebuilt C# systems (GameSaveManager and GameLoadManager) in your project.
Below is a full breakdown of how it works, including every variable, method, and usage.

🧠 Overall Purpose

This system:

- Adds custom Unity Editor menu options
- Generates script files automatically
- Writes full source code into those files
- Saves them inside the selected folder in the Project window
- Refreshes Unity so scripts appear instantly

So instead of manually creating scripts, you click a menu and they are generated for you.

📦 Namespace: SaveCustomGame.Editor

This means:

- It only runs inside the Unity Editor
- It does NOT run in builds (game runtime)

It is strictly a development tool.

Class: CreateBaseScript

This class is:

- Not static, but behaves like a utility container
- Only used in the Editor
- Provides script generation tools

CORE FUNCTION: CreateScript()

This is the **engine that actually creates files**

---

baseName

- Name of the script file
- Example: GameSaveManager
- Becomes: GameSaveManager.cs

---

baseCode

- Full text content of the script
- This is written directly into the file

Think of it as:
"the entire script template"

---

selectedPath

- Default folder is: Assets
- If user has something selected in Project window:
  - It uses that location instead

---

selectedObject

- Whatever is currently selected in Unity Project window

Example:

- A folder
- A script
- A prefab

AssetDatabase.GetAssetPath()

- Converts Unity selection into a real file path

File.Exists check

Two protections:

1. If a script already exists → prevent overwrite
2. If selected item is a file → get its folder instead

---

scriptPath

- Final destination path of new script

Example:

- Assets/Scripts/GameSaveManager.cs

File.WriteAllText()

- Creates the file
- Writes the full script code inside it

This is the actual “generation step”.

📎 **AssetDatabase.Refresh()**

- Forces Unity to detect the new script immediately

Without this:

- Unity would not show the file instantly

📎 **AssetDatabase.LoadAssetAtPath()**

- Loads the newly created script as a Unity object

📎 **Selection.activeObject**

- Automatically selects the new script in Unity editor

So the user immediately sees it.

📎 **Debug.Log**

- Confirms success or failure in console

📂 **MENU SYSTEM SECTION**

This is where Unity buttons are created.

📎 **MenuItem Attributes**

These lines:

- Add buttons inside Unity Editor menu:
  - Assets → Create → Tools → Save Custom Game

So the user can click and generate scripts instantly.

🧩 **METHOD 1: CreateSaveManager()**

🎯 **Purpose**

Generates a full script called:
GameSaveManager

📎 **code variable**

This is a **big string containing an entire C# file**
It includes:

- Player save system
- Object save system
- Save keys
- Save logic
- Reset system

So this method literally generates a full system file.

📎 **Usage flow**

When user clicks menu:

1. String is created
2. Passed to CreateScript()
3. File is written to project
4. Unity refreshes
5. Script appears ready to use

🧠 **What the generated script does (conceptually)**

Inside the generated file:
It defines:

- Player position saving

- Player rotation saving
- Object active state saving
- Reset save system
- Save/load toggles

Key groups:

Player keys

- Position
- Rotation

Object keys

- State1
- State2

Game state

- Whether saving is allowed

Main features:

- Save player transform
- Load player transform
- Save object active states
- Reset everything

🧩 **METHOD 2: CreateLoadManager()**

🎯 **Purpose**

Generates:
GameLoadManager script

📌 **code variable**

Another full script string, but this one is a MonoBehaviour.

🧠 **What it does**

This generated script:

- Connects player object
- Loads saved data
- Saves data through GameSaveManager
- Reloads scene
- Handles editor reset button

📌 **Serialized fields**

player

- Reference to player GameObject

gameObjects

- List of objects to save/load state

OnGameRestart

- Event triggered after scene reload

📌 **Properties**

Provide controlled access to:

- Player
- GameObjects list

📌 **Start()**

When scene starts:

1. Checks if player exists
2. Checks object list

3. If save allowed:
    - loads data automatically
4. Otherwise logs message

📎 **SaveGame()**
Saves everything:
- Player position + rotation
- Object active states
- Enables loading flag

📎 **LoadGame()**
Restores everything:
- Player position + rotation
- Object states
- Resets time scale
- Resets audio

📎 **ReloadScene()**
- Reloads current Unity scene
- Triggers restart event

📎 **ReloadSceneTime()**
- Delayed version of reload
- Uses Invoke system

📎 **Custom Inspector (Editor-only)**
Adds a button inside Inspector:
"Clear Save Data"
When clicked:
- Calls ClearAllSaveData()
- Resets everything to default state

🎮 **How the system is used in Unity**
Step 1
User right-clicks in Project window
Step 2
Selects:
- Create → Tools → Save Custom Game

Step 3
Chooses:
- GameSaveManager OR GameLoadManager

Step 4
Unity generates full script automatically

🚀 **Summary**
This script is a:
Code generator for your save system
It provides:
⚙️ **Editor automation**
- Menu buttons
- Script creation tools

📂 **File generation**
- Writes full C# files automatically

🧠 **Prebuilt systems**

- Save manager
- Load manager

🎮 **Unity integration**

- AssetDatabase refresh
- Scene selection
- Inspector customization

Prefab Creator

This script is a **Unity Editor utility that automatically creates and places UI prefabs (Save / Load menus) into your scene**. It works as a bridge between the project's prefab assets and the current Unity scene, making UI setup almost one-click.
It is entirely editor-side tooling — nothing here runs in the final game build.

🧠 **Overall Purpose**

This system:

- Finds UI prefabs inside the project
- Ensures a UI Canvas exists in the scene
- Instantiates the prefab into the scene
- Automatically parents it correctly
- Fixes it for editing (unpacks prefab)
- Selects it for the user
- Enables instant renaming
- Adds undo support everywhere

📦 **Namespace: SaveCustomGame.Editor**

This means:

- Only works inside Unity Editor
- Not included in builds
- Used for development workflow tools

🧰 **Class: PrefabCreator**

This is a **static utility class**, meaning:

- You never create an instance of it
- It only contains helper functions
- It is triggered by Unity menu commands

🎨 **CORE SECTION: Canvas And Prefab Creation**

📦 **Method: CreateUICanvas()**

🎯 **Purpose**

Creates a UI Canvas automatically if one does not exist.

📎 **canvasGO**

- A new GameObject named "Canvas"
- This becomes the root UI container

📎 **canvas (component)**

This object gets UI components:

- Canvas → renders UI
- CanvasScaler → handles screen scaling
- GraphicRaycaster → allows UI clicks

⚙ **Canvas configuration**
It sets:
- Render mode → screen overlay (UI on screen)
- Layer → UI layer
- Sorting order → 0 (default UI priority)
- Target display → main screen

🖇 **EventSystem handling**
What it does:
- Checks if an EventSystem already exists
- If not, it creates one

Why important:
UI in Unity cannot work without:
- EventSystem
- Input Module

So it ensures:
✓ UI clicks work
✓ Buttons function properly

🖇 **Undo.RegisterCreatedObjectUndo**
This allows:
- Ctrl+Z support in Unity Editor

So if user creates something by mistake:
→ they can undo it instantly

🖇 **return canvas**
Returns the created Canvas so other functions can use it.

🔍 **Method: FindPrefabByName()**
🎯 **Purpose**
Finds a prefab inside the project by name.

🖇 **prefabName**
- Name of the prefab to search for
- Example: “Save Menu (TMP)”

🖇 **AssetDatabase.FindAssets**
Search behavior:
- Looks through all prefabs in project
- Filters by name and type

🖇 **guid loop**
Each GUID represents a found asset.
For each one:
Converts:
- GUID → file path

🖇 **filtering logic**
It checks:
- Must be inside “Save Custom Game/Prefab”
- Must match exact prefab name (case insensitive)

This avoids:

- Wrong duplicates
- Incorrect assets

📎 **LoadAssetAtPath**

- Loads prefab as usable GameObject

❌ **if not found**

- Logs error
- Returns null

🧩 **Method: CreateAndConfigurePrefab()**

🎯 **Purpose**

This is the **main execution function**.
It handles:

- Canvas creation
- Prefab loading
- Instantiation
- Parenting
- Final setup

📎 **fileName**

- Prefab name to load

📎 **selectedGameObject**

- Optional parent object from hierarchy
- If null → UI goes into Canvas

📎 **canvas lookup**

- Tries to find existing Canvas
- If none → creates one automatically

📎 **prefab loading**

- Uses FindPrefabByName()
- If missing → stops execution

📎 **parent selection logic**

If user selected something:

- UI becomes child of that object

Else:

- UI is placed under Canvas

📎 **PrefabUtility.InstantiatePrefab**

- Instantiates prefab in the scene
- Keeps prefab connection initially

📎 **FinalizePrefabSetup()**

After creation:

- Performs cleanup and usability setup

✨ **Method: FinalizePrefabSetup()**

🎯 **Purpose**

Makes the instantiated prefab:

- Editable
- Selectable
- Easy to rename

📎 **Undo.RegisterCreatedObjectUndo**

- Allows undo for prefab creation

📎 **UnpackPrefabInstance**

This is important:

- Removes prefab link
- Makes it fully editable in scene

So now it becomes:
a normal GameObject, not a locked prefab instance

📎 **Selection.activeGameObject**

- Automatically selects the object in hierarchy

So user immediately sees it.

🖥️ **Auto rename feature**

This part:

- Waits one frame
- Then simulates F2 key press

Effect:
✓ Renaming mode opens instantly
✓ User can rename UI immediately

🧩 **MENU SYSTEM SECTION**

These are Unity Editor menu buttons.

🎮 **Legacy UI Prefabs**
📎 **CreateLoadMenuPrefab()**
📎 **CreateSaveMenuPrefab()**
Menu paths:

- GameObject → Tools → Save Custom Game → UI → Legacy

What they do:

- Create older UI versions of:
  - Load Menu
  - Save Menu

Flow:

1. User clicks menu option
2. Calls CreateAndConfigurePrefab
3. Loads correct prefab name
4. Instantiates into scene

🎮 **TMP UI Prefabs (TextMeshPro)**
📎 **CreateLoadMenuPrefabTMP()**
📎 **CreateSaveMenuPrefabTMP()**
Menu paths:

- GameObject → Tools → Save Custom Game → UI

These create:

- Modern UI versions using TMP

🚀 HOW THE SYSTEM IS USED

Step 1
User right-clicks in Unity Hierarchy or top menu
Step 2
Selects:
- Save Menu (TMP)
- Load Menu (Legacy)
- etc.

Step 3
System automatically:
- Finds prefab
- Creates Canvas if needed
- Instantiates UI
- Parents correctly
- Selects object
- Enables rename mode

⚙️ KEY VARIABLES SUMMARY

canvasGO
- Root UI GameObject

canvas
- UI rendering system component

existingEventSystem
- Detects if UI input system already exists

guids
- Search results from Unity asset database

path
- Converted file location of prefab

prefab
- Loaded UI prefab asset

parent
- Where UI will be placed in scene

instance
- Final created UI object

🚀 FINAL SUMMARY

This script is a:
One-click UI system generator for Unity Save/Load menus
It handles everything automatically:

🧠 **Smart setup**
- Creates Canvas if missing
- Ensures EventSystem exists

📦 **Asset management**
- Searches prefab database
- Loads correct UI assets

🎮 **Scene integration**
- Instantiates UI directly into scene
- Parents it properly

✨ **Developer experience**
- Auto-selects object
- Auto rename mode

- Full undo support

Save Custom Settings Provider

This script is a **fully custom Unity Project Settings editor for your Save Custom Game system**. It replaces the default inspector for your save configuration asset and turns it into a structured, tab-based configuration panel with advanced UI behavior.
In simple terms:
It creates a “control panel inside Unity Settings” where you manage how your save system behaves, what data it stores, and how it is organized.

Overall Purpose

This system is responsible for:

- Displaying SaveCustomObject settings inside Project Settings
- Allowing full editing of save system configuration
- Organizing data in tabs (Float, Int, Vector, etc.)
- Managing autosave behavior
- Choosing storage type (Game Data / LocalLow / PlayerPrefs)
- Editing custom save structure dynamically
- Persisting editor state (tab memory)
- Ensuring asset exists and can be created automatically

Namespace: SaveCustomGame.Editor

This means:

- Editor-only system
- Does NOT exist in runtime builds
- Used purely for Unity editor tooling

Class: SaveCustomSettingsProvider

This is the core of the system.
It is:

- Static → no instances needed
- Registered into Unity Project Settings
- Responsible for drawing a full custom UI panel

Cached Variable: data

Purpose
Stores reference to:
SaveCustomObject (your main save configuration asset)
Behavior

- Loaded once
- Reused every time settings window opens

If null:

- It tries to load automatically from SaveDataUtility

MAIN ENTRY POINT: CreateProvider()

This is the function Unity uses to register your settings tab.

SettingsProvider("Project/Save Custom Game")

This defines:

- Where the settings appear:
  - Project Settings → Save Custom Game

label = "Save Custom Game"

- Name shown in Unity UI sidebar

guiHandler

This is the **entire UI drawing logic**
Everything inside this block is what you see in the settings window.

SECTION: Missing Asset Handling

data == null check

If the SaveCustomObject does not exist:
UI shows:

- Warning message
- Button to create asset

Create Settings Asset button

When clicked:

1. Calls asset creation system
2. Reloads data reference
3. Highlights asset in Project window
4. Selects it automatically
5. Exits GUI safely

Why ExitGUI matters

Prevents Unity layout errors after asset creation.

SECTION: Asset Reference Display

This part shows the actual SaveCustomObject in UI.

ObjectField (disabled)

- Displays asset reference
- Cannot be edited directly

Ping button

- Highlights asset in Project window

Open button

- Opens asset in inspector

SECTION: SerializedObject (so)

SerializedObject

This is Unity's safe editing system.
It allows:

- Undo support
- Prefab-safe editing
- Proper serialization updates

so.Update()

- Syncs latest data before editing

Cached Properties

These are references to fields inside SaveCustomObject:
pixelLimit

- Screenshot resolution limit

saveGameByEvent
- Enables event-based saving

saveGameByTime
- Enables time-based saving

saveInterval
- Time between autosaves

gameData / localLow / playerPrefs
- Storage mode flags

saveCustomItems
- Main list of custom save entries

📸 **SECTION: Screenshot Settings**

pixelLimit

Controls:
- Maximum resolution for screenshots
- Prevents memory overload

💾 **SECTION: Auto Save Settings**

saveGameByEvent
- Save triggered manually or by events

saveGameByTime
- Enables automatic timed saving

---

saveInterval
- Only editable if time-based saving is active

Behavior:
- Disabled when toggle is off
- Enabled when toggle is on

💾 **SECTION: Storage Mode**

This is a **radio-button system (toolbar)**.

Options:

1. Game Data
- Saves inside game directory

2. LocalLow
- Uses Unity persistent data path

3. PlayerPrefs
- Lightweight storage system

📎 **selectedIndex**

Determines which option is active.

📎 **GUILayout.Toolbar**
- Displays selectable buttons
- Only one can be active

📎 **switch text**

Displays description based on selection:
- Explains chosen storage method

🧩 **SECTION: Custom Data System**

This is the most complex part.

It allows:
Fully dynamic creation of save data structures inside the editor.

saveCustomItems
This is a list of:
- Custom save groups (like PlayerStats, Inventory, etc.)

Each item contains:
- Vectors
- Floats
- Ints
- Strings
- Bools

Item Controls
+ button
- Adds new item

- button
- Removes last item

ITEM LOOP
Each save item is drawn individually.

itemTag
- Name of the data group
- Example: "Player"

itemVector / itemFloat / etc.
Each represents a different data type container:
- Vector → positions, rotations
- Float → numeric values
- Int → counters
- String → text
- Bool → flags

SECTION: Tabs System
Each item has a **tab interface**:
Tabs:
- Vector
- Float
- Int
- String
- Bool

tabIndex (EditorPrefs)
- Saves last selected tab per item
- Persists even after closing Unity

dynamic labels
Tabs show:
- Name + element count
- Example: "Float (3)"

tooltip system

Each tab explains:

- What type of data it stores
- Use cases

📋 **ACTIVE LIST SELECTION**

Based on tab:

- Selects correct list automatically

➕ **ENTRY CONTROLS**

Each list has:

+ button

- Adds entry

- button

- Removes entry

🧩 **ENTRY LOOP**

Each entry contains:

tagProp

- Key name (identifier)

valueProp

- Stored value

🧠 **Dynamic field detection**

System checks type and chooses:

- Vector4 field
- Float field
- Int field
- String field
- Bool field

📦 **Vector handling**

Vector4 is drawn manually:

- Custom Unity field layout
- Ensures clean alignment

❌ **Remove button**

- Deletes individual entry

💾 **FINAL SAVE STEP**

so.ApplyModifiedProperties()

- Applies UI changes to asset

Undo.RecordObject

- Enables Ctrl+Z support

EditorUtility.SetDirty

- Marks asset as modified

AssetDatabase.SaveAssets

- Saves changes to disk

🔎 **SEARCH KEYWORDS**

At the bottom:

This adds search support in Project Settings:

- Save
- Autosave
- Persistence
- Save System

🚀 **FINAL SUMMARY**

This script is a:

Full custom Unity Project Settings UI for your save system

It provides:

🎛 **Control Panel UI**

- Organized settings page inside Unity

💾 **Save system configuration**

- Autosave toggles
- Storage mode selection

🧩 **Dynamic data editor**

- Add/remove custom save structures
- Multi-type support (Vector, Float, Int, etc.)

🧠 **Smart editor behavior**

- Tab memory (EditorPrefs)
- Undo support
- Asset auto-creation
- Safe serialization system

🧭 **In short**

This system turns your save system into:
a professional-grade Unity settings panel with dynamic data editing and structured UI organization.

Auto Save Custom

This script is an **automatic background saving system** for your Save Custom Game framework. It runs inside a Unity scene and continuously handles timed saves without player input, using a rotating slot system.

In simple terms:

It automatically saves the game every X seconds, cycles through save slots, and respects global save settings.

🧠 **Overall Purpose**

This component is responsible for:

- Running autosaves in the background
- Triggering saves based on time intervals
- Rotating between multiple save slots (1–6)
- Respecting global save configuration (enabled/disabled modes)
- Persisting progress using PlayerPrefs
- Working with SaveCustomInScene as the actual save executor

📦 **Class: AutoSaveCustom**

This is a **MonoBehaviour component**, meaning:

- It attaches to a GameObject in the scene
- It runs automatically during gameplay
- It uses Unity lifecycle methods (Start, FixedUpdate)

SERIALIZED FIELD SECTION

saveCustomInScene

Purpose:
Reference to the system that actually performs saving.
What it does:

- Connects this autosave system to the core save engine
- Allows calling SaveData()

Usage:
If not assigned manually:

- It is automatically retrieved from the scene using SaveDataUtility

PRIVATE FIELDS SECTION

These control internal autosave logic.

currentAutoSaveSlot

Purpose:
Tracks which save slot is currently being used.
Behavior:

- Starts at slot 1
- Increments after each autosave
- Cycles between 1 → 6

Why it exists:
Prevents overwriting the same save repeatedly.

timeSinceLastSave

Purpose:
Tracks how much time has passed since last autosave.
Behavior:

- Increases every physics frame
- Resets after save triggers

saveInterval

Purpose:
Defines how often autosave happens (in seconds)
Source:

- Loaded from SaveCustomObject settings

Example:

- 60 seconds → save every minute

saveKey

Purpose:
Key used in PlayerPrefs storage
What it stores:

- Current autosave slot index

Why:
Allows autosave to continue correctly after restarting the game

UNITY LIFECYCLE METHODS

Start()

This runs once when the object is created.

Step-by-step behavior:

1. Ensure save system reference exists
If missing:
- It fetches SaveCustomInScene automatically

2. Disable autosave initially
- Prevents accidental saves before system is ready

3. Load save interval
- Gets value from SaveCustomObject configuration

So autosave timing is centralized and configurable.

4. Load last used slot
- Reads PlayerPrefs
- Restores previous autosave slot index

Why this matters:
Even after restarting the game:
✓ autosave continues where it left off
✓ prevents overwriting same slot repeatedly

⏱ **FixedUpdate()**
This runs on a **fixed time step**, making it ideal for consistent timing.

📌 **time accumulation**
Each frame:
- Adds fixed delta time to counter

This creates a reliable timer system.

📌 **save trigger check**
When time exceeds interval:
Step 1:
- Reset timer

Step 2:
- Check if event-based saving is disabled

Step 3:
- If allowed → trigger SaveAutoGame()

Why FixedUpdate is used:
- Ensures consistent timing
- Not affected by frame drops
- Stable autosave intervals

⚙ **PUBLIC METHOD: SaveAutoGame()**
This is the **actual autosave execution function**.

📌 **autosaveEnabled check**
If autosave is disabled:
- Function immediately exits
- No saving occurs

📌 **fileName assignment**
It sets save file name dynamically:
- Format: “0 - X”

- X = current slot

Example:

- 0 - 1
- 0 - 2
- 0 - 3

🖇️ SaveData()

This calls the actual save system:

- Writes data to disk/storage
- Captures current game state

🖇️ slot rotation logic

After saving:

- Slot increments by 1
- Cycles back after 6

Formula:

- (currentSlot % 6) + 1

Result:

- 1 → 2 → 3 → 4 → 5 → 6 → 1

🖇️ PlayerPrefs.SetInt()

Stores updated slot index permanently:

- Ensures persistence between sessions
- Prevents reset on restart

🔄 SYSTEM BEHAVIOR SUMMARY

Every frame:

1. Timer increases
2. When threshold is reached:
    - Check settings
    - Trigger save if allowed
3. Save uses next slot
4. Slot is stored permanently

🧠 KEY VARIABLES SUMMARY

saveCustomInScene

- Link to main save system

currentAutoSaveSlot

- Rotating save index (1–6)

timeSinceLastSave

- Timer accumulator

saveInterval

- Time between saves (seconds)

saveKey

- PlayerPrefs key for persistence

🚀 FINAL SUMMARY

This script is a:

Background autosave manager with timed execution and rotating save slots

It provides:

⏱️ **Time-based automation**

- Saves game every configured interval

🔁 **Slot rotation system**

- Prevents overwriting same save
- Cycles through multiple slots

💾 **Persistent tracking**

- Uses PlayerPrefs to remember progress

🎮 **Integration with save system**

- Delegates actual saving to SaveCustomInScene

🧭 **In simple terms**

This system acts like:
A silent background worker that saves your game every X seconds into different slots, without interrupting gameplay.

Save Custom Initialization

This script is responsible for **bootstrapping the entire save system automatically when the game starts**, ensuring everything is properly created, connected, and persistent across scene changes.

Overall Purpose

When the game launches, this system:

- Checks if the save system already exists in the scene.
- Loads the main save configuration asset.
- Creates a dedicated runtime object for saving.
- Attaches required components.
- Links everything together.
- Makes it persistent across scene loads.
- Prevents duplicate initialization.

In short: it guarantees the save system is always ready without manual setup.

Namespace and Context

Everything belongs to a save system namespace, meaning it is part of a modular framework and should not interfere with unrelated game systems.
It also uses shared utility functions (like debug helpers) from an exception utility module.

Class Overview

This is not a normal gameplay component attached to objects.
Instead, it is a **startup initializer class** that runs automatically before the first scene loads.

Automatic Execution Method
RunGameInitialization

This is the core method of the system.
When it runs
It executes automatically:

- Before any scene is loaded
- At game startup
- Only once per session

Step-by-step behavior
1. Startup log

It prints a debug message showing initialization has started.
Purpose:

- Helps confirm system boot sequence in the console.

2. Duplicate prevention check

It searches the current runtime scene for an existing save system object.
If it finds one:

- It assumes the system is already initialized
- It stops immediately

Purpose:

- Prevents duplicate save managers
- Avoids conflicts or double saving

3. Load main save configuration
It requests the central save configuration asset using a utility method.
This asset contains:

- Save settings
- Storage mode
- Autosave configuration
- Custom data structure

If it fails:

- It logs an error
- It stops initialization

Purpose:

- The system cannot function without its configuration data

4. Create runtime save object
It creates a new hidden GameObject in the scene with a fixed name.
This object acts as the **core container for the save system**.
It is not part of any prefab or scene file.
Purpose:

- Keeps save system runtime-only
- Ensures consistent setup in every scene

5. Attach required components
Two components are added:
Save system scene controller
Responsible for:

- Saving and loading data
- Managing file operations
- Holding runtime save logic

Auto-save controller
Responsible for:

- Automatic periodic saving
- Timer-based save triggers
- Rotating save slots

6. Link components together
The system connects both components:

- The scene controller receives the loaded configuration asset
- The auto-save component receives a reference to the scene controller

Purpose:

- Establish communication between systems
- Ensure both operate on the same data source

7. Make object persistent
The runtime save object is marked so it is not destroyed when scenes change.
Purpose:

- Keeps save system alive across the entire game session

- Prevents resets when loading new levels

8. Final log message
A confirmation message is printed showing successful initialization.

Key Design Idea
This script acts like a **silent bootstrapper**:
- You never manually place it in a scene
- It runs automatically
- It builds everything dynamically
- It guarantees one stable save system instance

Usage in Practice
You do not call anything manually.
To use the system:
- Just start the game
- The initializer runs automatically
- Save system becomes available instantly

Summary
This script ensures:
- One single save system instance
- Automatic setup at runtime
- Safe prevention of duplicates
- Persistent data across scenes
- Fully self-initializing architecture

Save Custom In Scene

This script is the **core runtime save system handler**, responsible for tracking gameplay state, capturing screenshots, storing scene information, and saving/loading everything either to disk or PlayerPrefs depending on configuration.
It is designed to continuously update game state in real time and serialize it into a structured save file.

Overall Purpose
This component handles:
- Saving game state (player progress, scenes, custom data)
- Loading previously saved sessions
- Tracking in-game time
- Capturing screenshots
- Choosing storage method (file system or PlayerPrefs)
- Managing scene reconstruction after load
- Keeping a synchronized save data container

Main Component Overview
This system is attached to a runtime GameObject and works together with the central save configuration asset.
It is the **bridge between gameplay and persistent storage**.

Serialized Fields (Inspector-Visible Variables)
saveCustomObject
This is the main configuration and data container.
It stores:

- Save settings (storage type, autosave rules)
- Screenshot data
- Custom save lists
- Scene names

Usage:

- Acts as the central “brain” of the save system
- Shared across all save operations

gameTime
A formatted string representing gameplay duration.
Example format:

- 00:10:45 (10 minutes, 45 seconds)

Usage:

- Displaying playtime in save menus
- Stored in save file

fileName
Defines the name of the save file.
Usage:

- Used when writing JSON to disk or PlayerPrefs
- Also used as the key for PlayerPrefs storage

savePath
Stores the final resolved file location.
Usage:

- Used for reading and writing save files
- Automatically updated depending on platform and settings

sceneNames
A list containing all active scene names at the time of saving.
Usage:

- Restores multi-scene setups when loading
- Supports additive scene loading

sceneCamera
Camera used for screenshot capture.
Usage:

- Captures visual snapshot of the game state
- Automatically detected if not assigned

Private Fields
elapsedTime
A continuously increasing timer (in seconds).
Usage:

- Tracks total gameplay duration
- Used to generate formatted game time string
- Saved and restored between sessions

Public Methods
ResetSave
Purpose:
Resets all runtime save data.
What it does:

- Resets elapsed time to zero

- Clears screenshot data
- Resets formatted time string
- Clears stored scene list

Usage:

- Used when starting a new game
- Used when wiping save data

SaveData

Purpose:

Converts current game state into a saved file.

Step-by-step behavior:

1. Camera validation

If no camera is assigned, it tries to find one automatically.

2. Screenshot capture

Takes a screenshot using the active camera and stores it in the save object.

3. Create save structure

Builds a structured data container containing:

- Screenshot
- Game time
- Scene list
- Custom data entries
- Elapsed time

4. Convert to JSON

Transforms the structure into a JSON string.

5. Storage decision

Depending on settings:

If LocalLow mode

- Saves to persistent system folder

If PlayerPrefs mode

- Saves as a string inside Unity's key-value system

Otherwise (file mode)

- Saves inside project or build directory
- Uses different paths depending on editor or build environment

6. Directory handling

Ensures the folder exists before writing the file.

7. Write file

Saves JSON to disk.

8. Cleanup

Resets camera reference after saving.

Usage

- Called manually (save button)
- Called by autosave system
- Used during checkpoints

LoadData

Purpose:
Restores saved game state from storage.
Step-by-step:
1. Load source selection

- If PlayerPrefs is enabled → load from key
- Otherwise → load from file

2. Validation
If no data is found:

- Logs warning
- Stops execution

3. Deserialize JSON
Converts string back into structured data.

4. Apply data
Calls internal method to restore everything into the scene.

Usage

- Called on game startup
- Called when loading a save slot
- Used by load menu systems

Private Methods
Update
Runs every frame.
Responsibilities:

- Increases elapsed time
- Updates formatted time string
- Synchronizes scene list with current game state

Usage:

- Continuous background tracking system

LoadDataString
Purpose:
Applies loaded save data back into the game world.
What it restores:

- Screenshot
- Game time
- Scene list
- Custom saved data
- Elapsed time

Scene restoration logic
If scenes exist in save:

- First scene is loaded normally
- Remaining scenes are loaded additively

Usage:

- Restores multi-scene gameplay setups

UpdateSaveCustomObject
Purpose:
Keeps the save object synchronized with current runtime state.
What it does:

- Updates formatted game time
- Clears scene list
- Rebuilds scene list from active scenes
- Stores updated list back into save object

Usage:

- Ensures save data always reflects real-time game state

UpdateGameTime

Purpose:

Converts raw seconds into readable time format.

Example output:

- 01:23:45

Logic:

- Converts seconds → minutes → hours
- Formats into fixed 2-digit display

Usage:

- UI display
- Save file metadata

GetCamera

Purpose:

Automatically finds a valid camera for screenshots.

Priority order:

1. Player object camera
2. Camera inside player hierarchy
3. Main camera
4. Any camera in scene

If none found:

- Logs error

Usage:

- Ensures screenshots always work without manual setup

SaveCustomFile Structure

This is the actual **data container saved to disk or PlayerPrefs**.

Fields

screenshot

Binary image data of the captured frame.

Usage:

- Save preview thumbnails
- Load screen previews

gameTime

Stored formatted playtime.

sceneNames

List of scenes active during save.

Usage:

- Restore multi-scene setups

saveCustomItems

Custom user-defined save data.

Usage:

- Player stats
- Inventory

- World state

elapsedTime
Raw gameplay time in seconds.

Key Design Concept
This system is built around three layers:
1. Runtime tracking
- Time
- Scenes
- Camera

2. Serialization layer
- Converts everything to JSON

3. Storage layer
- File system OR PlayerPrefs

Summary
This script:
- Continuously tracks gameplay state
- Automatically manages time and scenes
- Captures screenshots for save previews
- Serializes everything into structured data
- Saves/loads from multiple storage systems
- Reconstructs scenes and game state on load
- Works seamlessly with autosave and initialization systems

Save Custom Object

This script defines the **core data foundation of the entire save system**. Instead of handling saving logic directly, it acts as a structured container that stores everything the system needs: settings, screenshots, scene data, and custom user-defined values.
It is built as a **ScriptableObject-based database**, meaning it lives as an asset inside Unity and persists as structured configuration data.

Overall Purpose
This system is responsible for:
- Storing save system configuration
- Holding runtime save data (screenshots, time, scenes)
- Managing autosave settings
- Defining storage mode (file, LocalLow, PlayerPrefs)
- Providing a flexible custom data structure system
- Allowing creation of the asset directly from the Unity Editor

In simple terms:
👉 This is the “brain data container” of the save system.

Main ScriptableObject (Core Data Asset)
SaveCustomObject
This is the central asset that everything else references.
It contains multiple groups of data:

Screenshot and Scene Settings
screenshot
Stores image data as raw bytes.
Usage:

- Used for save thumbnails
- Used for previewing saved games

pixelLimit
Defines maximum screenshot resolution.
Usage:
- Prevents excessive memory usage
- Controls performance cost of screenshots

gameTime
Stores formatted playtime (example: 01:23:45)
Usage:
- Display in save/load menus
- Stored in save file metadata

sceneNames
List of scene names active during save.
Usage:
- Restores multi-scene setups
- Rebuilds game state when loading

Auto Save Settings
autosaveEnabled
Turns autosave system on or off.
Usage:
- Master switch for automatic saving

saveGameByEvent
Determines if saving happens via triggers (events).
Usage:
- Used when saving is manually controlled by gameplay events

saveGameByTime
Enables time-based autosaving.
Usage:
- Used by timer system (like AutoSaveCustom)

saveInterval
Time interval between autosaves (in seconds).
Usage:
- Controls how frequently autosave happens

Storage Mode Settings
These control where save data is stored.

gameData
Enables saving inside game folder structure.
Usage:
- Useful for development or portable builds

localLow
Uses system persistent storage location.
Usage:
- Recommended for production builds

playerPrefs
Stores data using Unity's key-value system.
Usage:

- Lightweight saves (settings, small data)

Custom Data System
saveCustomItems
A list that stores user-defined structured data.
Usage:

- Player stats
- Inventory systems
- World states
- Custom gameplay variables

This is the most flexible part of the system.

SaveCustomItem (Data Group Container)
This class acts like a **category container**.
It groups multiple data types under a single identifier.

itemTag
Unique identifier for this group.
Usage:

- Used to access this item from code

Data Lists
Each item can store multiple types:
itemVector
Stores 3D/4D data (positions, rotations, scales).

itemFloat
Stores decimal values.
Usage:

- Health, speed, timers

itemInt
Stores whole numbers.
Usage:

- Levels, counters, IDs

itemString
Stores text values.
Usage:

- Names, dialogue, identifiers

itemBool
Stores true/false values.
Usage:

- Switches, states, flags

Value Containers (Individual Data Entries)
Each of these represents a **single stored value with a tag**.
They all follow the same structure:

- A string identifier (tag)

- A value of a specific type

SaveCustomVector
Stores Vector4 data.
Usage:
- Position
- Rotation
- Custom vector data

SaveCustomFloat
Stores float values.
Usage:
- Health
- Speed
- Progress

SaveCustomInt
Stores integer values.
Usage:
- Score
- Level
- Count

SaveCustomString
Stores text values.
Usage:
- Player names
- IDs
- Saved notes

SaveCustomBool
Stores true/false values.
Usage:
- State toggles
- Conditions
- Unlock flags

Editor Utility Section (Only in Unity Editor)
This part only runs inside Unity Editor.
SaveCustomObjectCreator
This provides a menu option in Unity:
👉 Assets → Create → Tools → Save Custom Game → Save Custom Object Data

CreateCustomObjectData Method
This method is responsible for creating the save system asset.
Step-by-step behavior:

1. Search for base folder
It searches the project for a folder named:
- “Save Custom Game”

If found:
- It uses that location

2. Fallback creation

If not found:

- It creates a new folder inside Assets

3. Ensure Resources folder exists

Creates a subfolder:

- Resources

Usage:

- Required so the system can load the asset at runtime

4. Define asset path

Final location becomes:

- Save Custom Object Data.asset

5. Check if asset already exists

If it exists:

- It asks the user if they want to replace it

If user says no:

- It selects the existing asset instead

6. Create new asset

If approved:

- A new SaveCustomObject asset is created

7. Save and refresh project

Unity updates the asset database so it becomes available immediately.

8. Focus asset

Automatically selects the asset in the Project window.

9. Log confirmation

Prints confirmation message showing where it was created.

Key Design Concept

This system is built as a **data-driven save architecture**:
Instead of hardcoding save fields in scripts:

- Everything is stored in one flexible asset
- Data is grouped by tags
- Supports multiple types dynamically
- Can be expanded without modifying core code

Usage in the Game System

This asset is used by:

- SaveCustomInScene → writes and reads data
- AutoSaveCustom → reads settings like interval and enabled state
- Initialization system → loads it at startup
- Editor tools → configure and create it

Summary

This script defines:

- A central save configuration asset
- A flexible multi-type data system
- Structured storage for gameplay state
- Autosave and storage configuration
- Screenshot and scene tracking system

- Unity Editor tools to generate and manage the asset

Check Button Selected

This script is a small UI “safety system” that keeps keyboard/controller navigation working in Unity menus by always ensuring something selectable is focused. It runs continuously and automatically restores selection when the UI loses focus.

General Purpose

In Unity UI systems, especially when using keyboard or gamepad navigation, the system relies on a “currently selected UI element”. If nothing is selected, navigation breaks (pressing arrows or D-pad does nothing).
This script fixes that by:

- Watching the current UI selection every frame.
- If nothing is selected (or the selected object becomes inactive), it forces a default button to be selected again.

Variables

▶ defaultButton

This is the **fallback UI button**.

- It is assigned in the Inspector.
- It represents the button that should always regain focus if UI navigation is lost.
- Example usage: “Play”, “Start”, or “Main Menu” button.

If the system detects no valid selection, this button is automatically selected.

Property

▶ DefaultButton

This is just a safe way to access or change the default button from other scripts.

- You can read it (get).
- You can replace it at runtime (set).

Usage example idea:

- Changing default focus depending on which menu is open.

Methods

▶ Update (runs every frame)

This is the core logic of the script.
Every frame it does the following:

1. Check if UI system exists

- If there is no UI event system in the scene, the script stops.
- This prevents errors in scenes without UI.

2. Check current selection

It verifies:

- If nothing is selected in the UI system, OR
- If the currently selected object is inactive (disabled or hidden)

Both cases mean navigation is broken.

3. Restore UI focus

- If selection is invalid, it calls the default button and forces it to become selected.
- This immediately restores controller/keyboard navigation.

Usage

✓ Where to use it

- Main menus
- Pause menus

- Settings menus
- Any UI navigation-heavy screen

✓ How to set it up

1. Add this script to any UI manager object.
2. Drag a UI Button into the **Default Button** field in the Inspector.
3. Make sure your scene has an Event System (Unity usually creates one automatically).

✓ What it fixes

Without this script:

- UI navigation can “break” after closing panels.
- Controller input may stop working until mouse interaction happens.
- Selection can disappear when switching menus.

With this script:

- UI always “snaps back” to a valid button.
- Navigation stays consistent and predictable.

🧠 **Summary**

This is a **UI focus recovery system**:

- Monitors UI selection every frame.
- Detects when selection is lost or invalid.
- Automatically restores focus to a default button.
- Ensures smooth keyboard/controller navigation in menus.

Default Selected Button

This script is a small UI helper that ensures a specific button is automatically focused whenever a menu or UI object becomes active. It is mainly used to guarantee consistent keyboard and controller navigation in Unity menus.

🎯 **General Purpose**

When a UI panel is enabled (for example, opening a menu), Unity does not always automatically select a button. That can break navigation because:

- The EventSystem might still reference an old selection.
- No button may be currently highlighted.
- Controller/keyboard input may appear “dead”.

This script fixes that by:

- Clearing any previous UI selection.
- Forcing a specific button to become selected immediately when the UI becomes active.

📦 **Variables**

▶ **button**

This is the **default UI button reference**.

- Assigned in the Inspector.
- Represents the button that should always be selected when this UI object becomes active.
- Usually something like:
  - “Continue”
  - “Play”
  - “First option in menu”

This ensures the UI always starts in a predictable state.

🔗 **Properties**

▶ Button (getter/setter)
This is an access layer for the private button field.
It allows:

- Reading which button is currently assigned.
- Changing the default button dynamically from other scripts.

Usage example conceptually:

- Switching default selection depending on which submenu is opened.

⚙ **Methods**
▶ OnEnable (core behavior)
This method is automatically triggered when:

- The GameObject becomes active in the scene.
- The UI panel is shown.

It performs two key actions:

---

1. Clear previous selection
It tells the UI system:

- "Forget whatever was previously selected."

This prevents issues like:

- Old menu buttons staying highlighted.
- Invalid references to disabled UI elements.
- Broken navigation states.

---

2. Select the default button
After clearing selection:

- It immediately selects the assigned button.

This ensures:

- UI always has a valid starting focus.
- Keyboard and controller navigation works instantly.
- The user doesn't need to click before using input.

🎮 **Usage**
✓ Where to use it

- Main menus
- Pause menus
- Settings panels
- Any UI screen that appears/disappears

✓ How to use it

1. Attach this script to a UI panel or menu object.
2. Assign a UI Button to the "button" field in the Inspector.
3. Ensure the scene has an Event System (Unity usually adds this automatically).

🧠 **What it improves**
Without this script:

- UI might open with no selected button.
- Controller navigation might not work immediately.
- Users may need to click manually first.

With this script:

- Every time the UI appears, a button is already selected.
- Navigation is instantly ready.
- Menu behavior becomes predictable and consistent.

📌 **Summary**

This script is a **UI initialization helper** that:

- Runs when a UI object becomes active.
- Clears any previous UI selection.
- Forces a specific button to be selected.
- Ensures smooth controller/keyboard navigation in menus.

Paged Menu Base / Paged Menu Base TMP

This script is not a simple UI controller — it is a **base system for paginated save/load menus**, designed to be extended. It handles navigation, slot rendering, confirmation flow, and memory cleanup, while leaving the actual save/load logic to child classes.
Think of it as a **framework for save slots UI**, not a complete system by itself.

🧠 **General Idea**

The system creates a menu like this:

- Pages of save slots (page 0, 1, 2, etc.)
- Each page shows multiple slots (default 6)
- Each slot can display:
  - A button
  - A screenshot preview
  - A label (name + info)
- Supports:
  - Next / previous page buttons
  - Manual page input
  - Confirmation popup (overwrite/load)
  - Slot interaction handled by child classes

📦 **Core Data Structure**

▶ **SlotUI (one save slot)**

Each slot contains:

- **button**
  - The clickable UI button for that save slot.
  - Triggers slot actions (save or load depending on implementation).
- **image**
  - A visual preview of the save (usually a screenshot).
- **label**
  - Text showing:
    - page number
    - slot index
    - sometimes save metadata (like time)
- **path**
  - File path of the save slot (used when loading from disk).

👉 This structure allows each slot to behave like a full save file entry.

🧩 **Inspector Fields (UI configuration)**

▶ **Confirmation system**

- **confirmationPanel**
  - A UI panel that appears when confirming actions (like overwrite or load).
- **confirmButton**
  - Executes the final action after confirmation.
- **cancelButton**
  - Closes the confirmation panel.

👉 Used to prevent accidental overwrites or loads.

▶ **Page system settings**

- **maxSavePages**
  - Limits how many pages exist.
  - If set to -1 → unlimited pages.

▶ **Title system**

- **titleLoad**
  - UI text showing current mode (Save / Load / Autosave page).
- **text**
  - Label for normal pages (“Page”).
- **textAutomatic**
  - Label for page 0 (autosave page).

▶ **Slot array (main UI)**

- **slots**
  - Array of 6 slot UI elements.
  - Represents visible save slots on screen.

▶ **Navigation controls**

- **right**
  - Button to go to next page.
- **left**
  - Button to go to previous page.
- **inputField**
  - Allows typing a page number directly.

🔒 **Internal variables**

- **saveCustomInScene**
  - Reference to the save system backend.
  - Used to load/save actual data.
- **currentNumber**
  - Current page index (0, 1, 2, etc.).
- **initializedUI**
  - Ensures UI setup only happens once.
- **prefsKeyBase**
  - Key used to store last page in memory (PlayerPrefs).

🔗 **Properties**

Each inspector field has a public wrapper so child classes or other scripts can safely access and modify them.
Examples:

- ConfirmationPanel → control popup UI
- Slots → modify slot list dynamically
- MaxSavePages → adjust pagination rules at runtime

👉 This is done to support inheritance-based customization.

⚙ **Lifecycle Methods**

▶ **OnEnable**

Runs when the menu becomes active.
It:

1. Gets the save system reference.

2. Restores last opened page (from memory).
3. Initializes UI (buttons, slots).
4. Sets initial page state.
5. Updates title text.

👉 This ensures the menu always opens in a consistent state.

▶ **OnDisable**

Runs when menu closes.

- Calls cleanup function to destroy slot textures
- Prevents memory leaks from screenshot images

📦 **UI Setup System**

▶ **SetupUI**

Runs once.

It:

- Binds navigation buttons (left/right)
- Configures input field validation
- Links cancel button
- Assigns slot click events dynamically

👉 This builds all interaction logic automatically.

▶ **SetupInitialPage**

- Sets input field text to current page
- Loads first set of slots

📄 **Title logic**

▶ **UpdateTitle**

- If page = 0 → shows “Autosave”
- Otherwise → shows normal page label

👉 This visually separates autosave slot from normal saves.

🔁 **Page Navigation**

▶ **IncreasePage**

- Reads page number from input field
- Increases page
- Respects max limit
- Refreshes UI

▶ **DecreasePage**

- Decreases page
- Never goes below 0
- Refreshes UI

▶ **OnEndEditInputField**

- Called when user manually types page
- Validates range
- Forces update

▶ **ValidateNumericInput**

- Blocks non-number input
- Ensures only digits are allowed

📜 **Slot Refresh System**

▶ **RefreshSlots (core UI update)**
This is one of the most important methods.
It:

1. Validates page range
2. Saves current page in memory
3. Enables/disables navigation buttons
4. Updates slot labels
5. Clears old screenshots
6. Loads new slot data
7. Updates title

👉 This is the “UI redraw engine”.

▶ **LoadSlotsData (abstract)**
This is **not implemented here**.
Child classes must define:

- How save data is loaded
- How slots are filled

👉 This makes the system reusable for:

- Save menu
- Load menu
- Autosave menu

🎯 **Slot Interaction System**
▶ **OnSlotPressed**

- Called when a slot button is clicked
- Validates slot index and state
- Calls child method:
  → HandleSlotAction

▶ **HandleSlotAction (abstract)**
Child decides:

- Save into slot?
- Load from slot?
- Open confirmation?

▶ **ConfirmAction (abstract)**
Called after user confirms action.
Examples:

- overwrite save
- load save file

▶ **ShowConfirmationForSlot**

- Opens confirmation panel
- Connects confirm button dynamically
- Ensures correct slot is confirmed

▶ **OnCancelPressed**

- Closes confirmation UI

🧩 **Slot helpers**
▶ **SetSlotActive**
Controls slot appearance:

- Enables/disables button
- Shows/hides image (fade effect)

▶ **GetSavePath**
Builds correct file path depending on system mode:
- PlayerPrefs → no file path
- LocalLow → persistent folder
- File system → editor or build path

▶ **ApplySlotData**
Applies loaded save data to UI:
- Updates label with time
- Shows screenshot preview
- Activates slot visually

🧹 **Cleanup**
▶ **ClearSlotTextures**
Prevents memory leaks by:
- Destroying old textures
- Clearing references

👉 Important for screenshot-heavy systems.

🎮 **How to Use This Script**
This class is **not used directly**.
Instead:
You must create a child class that:
- Implements:
  - LoadSlotsData
  - HandleSlotAction
  - ConfirmAction

Example usage concept:
- Save Menu:
  - slot click → overwrite save
- Load Menu:
  - slot click → load game
- Autosave Menu:
  - slot click → restore autosave

🧠 **Final Summary**
This script is a:
✓ Paginated UI framework
✓ Save/load slot system base
✓ Confirmation system handler
✓ Navigation controller
✓ Screenshot UI manager
✓ Memory-safe UI renderer
But it does NOT define behavior itself — it delegates behavior to child classes.

Load Menu Manager / Load Menu Manager TMP

This script is a **specialized implementation of a paged save/load menu**, focused only on the **LOAD functionality**. It inherits all UI behavior, pagination, slot handling, and confirmation logic from the base paged system, and only defines *how loading actually works*.
Think of it as the “engine logic” plugged into a pre-built UI framework.

🧠 General Role of This Script

This script is responsible for:

- Reading saved data from disk or PlayerPrefs
- Showing that data inside UI slots
- Letting the player click a slot
- Asking for confirmation before loading
- Applying the selected save into the game

It does NOT handle UI layout or navigation — that is inherited.

📦 Main Inheritance Concept

This class extends a base system that already provides:

- Page navigation (next / previous / input field)
- Slot UI structure (buttons, images, labels)
- Confirmation panel logic
- Slot activation helpers
- Screenshot rendering utilities

👉 This script only defines **load-specific behavior**

⚙️ Methods

▶ OnEnable

What it does:

It simply calls the base system initialization.

Why it exists:

- Ensures all UI setup from the parent class runs correctly.
- Keeps this class consistent with the paged menu lifecycle.

Usage:

Automatically executed when the load menu is opened.

📂 Slot Loading System

▶ LoadSlotsData (core logic)

This is the **main function of the entire script**.

It runs every time:

- The page changes
- The menu refreshes
- Slots need to be updated

🔁 What it does step-by-step:

For each slot (usually 6):

1. Builds the save file name

It constructs a name like:

- “0 - 1”
- “2 - 5”

Based on:

- current page number
- slot index

👉 This ensures each page has its own save group.

2. Gets file path

It uses a helper method from the base system to determine:

- File system path OR
- PlayerPrefs key

3. Stores path in slot
Each slot remembers its own file location.
This is used later when loading.

4. Checks storage type
There are two possible systems:

🟡 **A) PlayerPrefs mode**
If the system uses PlayerPrefs:
- It checks if a key exists
- If NOT → slot is disabled
- If YES → data is loaded from string

Then:
- JSON is converted into usable save data
- Slot UI is updated with preview data

🟢 **B) File system mode**
If using files:
- It checks if file exists
- If NOT → slot is disabled
- If YES → file is read from disk

Then:
- JSON is parsed
- Slot is visually populated

5. ApplySlotData
If valid data exists:
- Shows screenshot preview
- Updates label text (page + slot + time)
- Enables the slot

🎯 **Result of this method**
After execution:
- Empty slots are disabled
- Filled slots show preview data
- UI reflects actual saved games

🎮 **Slot Interaction System**
▶ **HandleSlotAction (slot click logic)**
This runs when the user clicks a slot.

What it does:
1. Checks if slot has:
    - Image
    - Texture (preview screenshot)

👉 If not, it ignores the click.

2. If valid:
    - Opens confirmation panel

Why confirmation exists:
Prevents accidental loading of saves.
Example:

- User clicks slot → nothing happens immediately
- Confirmation appears → “Load this save?”

✓ Usage flow:
User clicks slot → system asks for confirmation

✅ Confirm Load Action
▶ ConfirmAction (final load step)
This is executed only after user confirms.

What happens:
1. Builds file name
Same format as before:
- page + slot number

2. Assigns file path
- Uses stored slot path
- Ensures correct save file is targeted

3. Loads data
Calls the save system:
- Reads save file
- Restores:
  - player position
  - scenes
  - game state
  - custom data

4. Closes UI
- Hides confirmation panel

🎮 Final Behavior Flow
Entire user interaction looks like this:
1. Menu opens
2. Slots are loaded from storage
3. User sees previews
4. User clicks slot
5. Confirmation appears
6. User confirms
7. Game state is loaded
8. Menu closes or updates

🧠 Key Variables (inherited, used indirectly)
Even though not declared here, this script depends on:
- slots → UI slot array
- inputField → current page input
- currentNumber → current page index
- saveCustomInScene → save system backend
- confirmationPanel → UI popup

🧩 Summary
This script is a **LOAD controller layer** for a modular save system:

✓ Reads save data from storage
✓ Displays it in paged UI slots
✓ Handles user interaction
✓ Requires confirmation before loading
✓ Applies save state into the game

Save Menu Manager / Save Menu Manager TMP

Below is a structured explanation of how the **Save Menu Manager (TMP)** script works, describing each part in terms of behavior, variables, and usage (without using C# syntax).

🧠 Overview

This script is a **specialized save menu system** built on top of a paginated UI framework.
Its job is to:

- Display save slots in pages.
- Show previews of saved data (screenshot, time, etc.).
- Allow the player to save into slots.
- Handle overwrite confirmations.
- Refresh UI after changes.

It extends a **base paged menu system**, meaning most navigation, pagination, and UI structure already exists. This script only defines **save-specific behavior**.

🧩 Core Variables (Inherited from Base)

Even though they are not re-declared here, this script heavily uses inherited data:

📦 Slots array

- Represents the visible save slots on screen (usually 6).
- Each slot contains:
  - Button (clickable area)
  - Image (screenshot preview)
  - Label (text info)
  - Path (file location)

Usage:
Used to display save files or empty slots depending on content.

📄 inputField

- Text input where the player can type a page number.

Usage:
Determines which “page” of saves is being shown.

🔢 currentNumber

- Current page index.

Usage:
Used to calculate which save slots are shown.

💾 saveCustomInScene

- Reference to the actual save system logic.

Usage:
Used to execute saving/loading operations.

⚠️ confirmationPanel

- UI panel that asks “Are you sure?” before overwriting saves.

Usage:
Shown when overwriting existing save data.

🔄 Main Methods

🎮 OnEnable

What it does:
When the menu becomes active:

- Runs base menu setup logic.
- Restores last opened page.
- Loads slot data.
- Updates the title text.

Usage:
Ensures the save menu always opens in the same state the player left it.

📦 LoadSlotsData (Main save-slot display logic)

What it does:
For every slot on the page:

1. Builds a file name based on:
    - current page number
    - slot index
2. Determines save path depending on storage mode:
    - File system OR PlayerPrefs
3. Checks if data exists:
    - If not → slot is marked empty
    - If yes → data is loaded
4. Converts stored data into a usable format
5. Calls a helper to apply:
    - screenshot preview
    - text label
    - slot activation

Usage:
This is what visually fills the menu with save data.

🖱️ HandleSlotAction (click on slot)

What it does:
When the player clicks a slot:

- Checks if slot already has data (occupied or empty)
- If occupied:
  → opens confirmation panel
- If empty:
  → not used here in load version

Usage:
Prevents accidental overwriting of saves.

⚙️ ConfirmAction (final confirmation step)

What it does:
When player confirms:

- Sets filename based on selected slot
- Loads data from that slot
- Closes confirmation UI

Usage:
Final step before loading saved game state.

🎨 SetSlotActive (UI control override)

What it does:
Controls visual behavior of slots:

- Buttons always stay clickable (important for save UI)
- Image transparency depends on whether a save exists

Usage:
Keeps UI consistent in save mode.

💾 **PerformSave (actual saving logic)**
What it does:

1. Builds file name based on:
    - current page
    - slot number
2. Calls save system to store data
3. Refreshes UI:
    - clears old previews
    - reloads updated slots

Usage:
This is the real "write save file" function.

✓ **ConfirmAction (save version)**
What it does:

- Calls PerformSave
- Closes confirmation panel

Usage:
Final overwrite confirmation step.

🧭 **How the Save Menu Works (Flow)**
1. Menu opens

- Loads current page
- Loads save slots

2. Player clicks a slot

- If empty → save immediately
- If occupied → ask confirmation

3. If confirmed

- Overwrites save
- Refreshes UI

4. UI updates

- New screenshot appears
- Labels update
- Slots refresh

🎯 **Key Design Idea**
This system separates:

- **Base logic (navigation, pagination)** → handled by parent class
- **Save logic (file writing, overwrite rules)** → handled here

So this script is basically the **"save behavior layer"** of the UI system.

Test Save Custom

Below is a structured explanation of how the **Test Save Custom** script works, focusing on variables, methods, and how it is used at runtime (without using C# syntax).

🧠 **Overview**
This script is a **testing and control utility for the save system**.

Its purpose is not gameplay logic, but rather:

- Turning auto-save on/off manually
- Triggering saves from UI buttons
- Testing save behavior at runtime
- Toggling GameObjects for debugging persistence

It is designed to be used mainly in:

- UI buttons
- Debug menus
- Editor/testing scenes

🧩 Variables

🎮 GameObjects array

What it is:
A list of scene objects that can be toggled on and off.
Usage:

- Used for testing runtime changes
- Helps verify if objects persist correctly after saving/loading

Example use:
You assign lights, enemies, UI elements, or props and toggle them during testing.

💾 testSaveSlot

What it is:
An optional number representing a save slot index used for testing.
Usage:

- Used only for logging/debugging
- Helps identify which slot was used during a save test
- Does NOT directly control saving logic (only informational here)

⚙️ Public Methods (UI / External Control)

These methods are meant to be connected to UI buttons or external triggers.

🚀 ActivateAndSave

What it does:

- Turns auto-save ON
- Immediately triggers a save

Usage:
Useful for:

- “Start saving system” button
- Debug quick-save test
- Testing auto-save behavior immediately

❌ DisableAutoSave

What it does:

- Turns auto-save OFF
- Prints a debug message confirming the change

Usage:
Useful for:

- Debugging save interruptions
- Testing manual-only saving behavior
- Temporarily disabling background saves

💾 TriggerSave

What it does:

- Calls the internal save function directly

Usage:

Used when:

- You want a manual save button
- You want to simulate autosave behavior
- You want to test saving without enabling auto-save

🧠 Internal Save Logic

💾 SaveTestSlot (core save trigger)

What it does:

1. Triggers a save event in the save system
2. Logs debug information:
    - If a slot number exists → shows slot index
    - If not → generic save message

Usage:

This is the **central save test function** used by multiple methods.

🔄 GameObject Testing System

🔁 SwitchGameObject

What it does:

1. Checks if any GameObjects are assigned
2. If none exist:
    - Prints warning message
    - Stops execution
3. If objects exist:
    - Loops through each one
    - Toggles its active state:
        - If active → becomes inactive
        - If inactive → becomes active
4. Prints confirmation message

Usage:

This is used for debugging:

- Testing if objects are saved/loaded correctly
- Simulating gameplay changes
- Verifying persistence behavior
- Checking scene state transitions

🧭 How the Script Works (Flow)

1. Setup phase

You assign:

- Objects to toggle
- Optional test slot number

---

2. Runtime interaction (via UI)

You can:

- Enable auto-save + trigger save instantly
- Disable auto-save
- Trigger a manual save
- Toggle GameObjects

---

3. Save system interaction
When saving is triggered:
- It calls the central save event system
- Logs information for debugging
- Optionally associates a slot index (for testing clarity)

4. Debug behavior
Every action produces logs like:
- Auto-save enabled/disabled
- Save triggered
- Slot used
- Object toggles

This makes it very useful during development.

🎯 **Key Purpose of This Script**
This script is basically a **developer control panel for the save system**, allowing:
- Manual testing without keyboard shortcuts
- UI-driven save control
- Runtime debugging of persistence
- Scene object state testing